第四部分

算法的基础理论

主讲人：孙歧峰

联系电话：13864717907
Q Q号码：274342509
邮　　箱：sunqf@upc.edu.cn

3.1 渐近符号（课后自己深入学习）

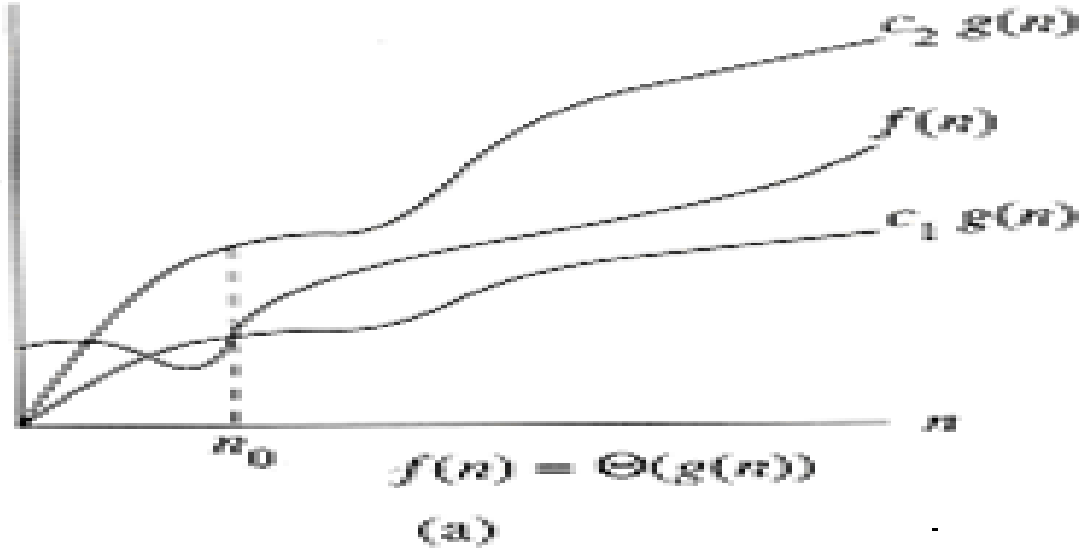
用来表示算法的渐近时间的记号是用定义在自然数N={ 0,1,2…}上的函数来定义的。因为T(n)一般仅定义在整数的输入规模上。

1. Θ记号—渐近确界(上下限:lower bound and upper bound)

$\Theta(g(n)) = \{ f(n)$: 存在正常数c_1, c_2和n_0，使对所有的$n \geq n_0$，有$0 \leq c_1.g(n) \leq f(n) \leq c_2.g(n) \}$

即，对于任意一个函数f(n)，若存在正常数c_1, c_2,使当n充分大时，f(n)能被夹在$c_1.g(n)$和$c_2.g(n)$中间，则f(n)属于集合$\Theta(g(n))$ 。

因为$\Theta(g(n))$是一个集合，所以可以写成“ $f(n) \in \Theta(g(n))$ ”。但是人们习惯写成“$f(n) = \Theta(g(n))$”来表示相同的意思！



3.1 渐近符号

例1：证明　$n^2-3n=\Theta(n^2)$

要证明存在常数c_1,c_2和n_0，使得对所有的$n\geq n_0$，有

$$c_1n^2\leq \frac{1}{2}n^2-3n\leq c_2n^2 \quad \text{成立}$$

用n^2除以不等式得：

$$c_1\leq \frac{1}{2}-\frac{3}{n}\leq c_2$$

于是，$c_2\geq \frac{1}{2}-\frac{3}{n}$　得知：$c_2\geq\frac{1}{2}$　时对所有$n\geq 1$成立

同样可知，$c_1\leq \frac{1}{2}-\frac{3}{n}$　在　$c_1\leq\frac{1}{14}$　时对所有$n\geq 7$成立

因此，取$c_1=\frac{1}{14}$　$c_2=\frac{1}{2}$　$n_0\geq 7$　有　$c_1n^2\leq \frac{1}{2}n^2-3n\leq c_2n^2$　成立！

例2：证明　$6n^3\neq\Theta(n^2)$

反证法：假设成立，则存在常数c_2和n_0使得对所有的$n\geq n_0$有 $6n^3\leq c_2n^2$

于是，$n\leq c_2/6$　由于c_2是常数，所以对于任意大的n是矛盾的！结论得证。

3.1 渐近符号

注意：

（1）Θ(g(n))中不同的函数，需要不同的常数！

（2）一个渐近正函数中的低阶项在决定渐近确界时可以被忽略，因为当n很大时它们就相对不重要了，最高阶项很小的一部分就足以超越所有的低阶项。

（3）一般可将c1取略小于最高阶项系数值，可将c2取稍大于最高阶项系数的值。

例3：证明　若 $f(n)=an^2+bn+c$ 其中a,b和c都是常数，且a>0　则 $f(n)=\Theta(n^2)$

取$c_1=a/4$，　$c_2=7a/4$

$n_0=2.\max((|b|/a, \sqrt{(|c|/a)})$

有，对于所有的$n\geq n_0$,$c_1.n^2\leq an^2+bn+c\leq c_2.n^2$ 成立

3.1 渐近符号

定理： 如果$f(n)=a_m n^m+ \ldots\ldots + a_1 n + a_0$ 且$a_m>0$,则$f(n)=\Theta(n^m)$

证明：略！

大Θ比率定理： 对于函数f(n)和g(n)，如果极限 $\lim_{n\to\infty} f(n)/g(n)$ 和 $\lim_{n\to\infty} g(n)/f(n)$ 都存在，则$f(n)=\Theta(g(n))$当且仅当存在正的常数c_1,c_2，使得 $\lim_{n\to\infty} f(n)/g(n) \le c1$ ，

$\lim_{n\to\infty} g(n)/f(n) \le c2$

例4 (1) f(n)=3n+2, g(n)=n (2) f(n)=10n²+4n+2, g(n)=n² (3) f(n)=6*2ⁿ+n², g(n)=2ⁿ

$\because \lim_{n\to\infty}(3n+2)/n = 3$

$\lim_{n\to\infty} n/(3n+2) = 1/3 < 3$

$\therefore$ $3n+2=\Theta(n)$

$\because \lim_{n\to\infty}(10n^2+4n+2)/n^2 = 10$

$\lim_{n\to\infty} n^2/(10n^2+4n+2) = 0.1 < 10$

$\therefore$ $f(n)=10n^2+4n+2=\Theta(n^2)$

$\because \lim_{n\to\infty}(6*2^n+n^2)/2^n = 6$

$\lim_{n\to\infty} 2^n/(6*2^n+n^2)/2 = 1/6 < 6$

$\therefore$ $f(n)=6*2^n+n^2=\Theta(2^n)$

3.1 渐近符号

2. O记号— (渐近上界:upper bound)

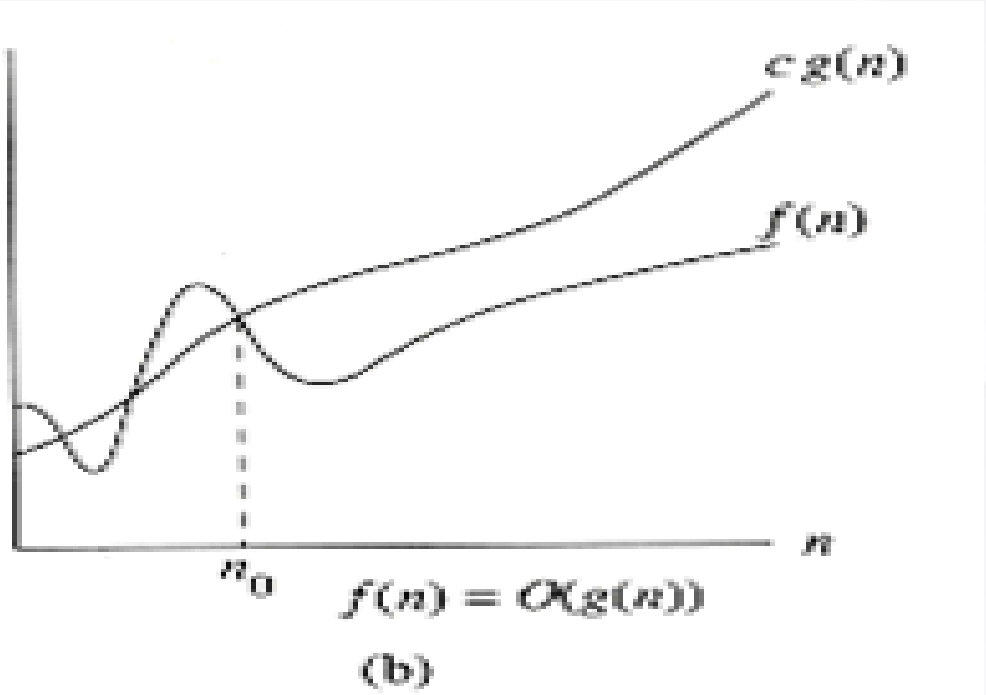
O记号定义增长率的上界——一般是指最小上界。

$O(g(n)) = \{ f(n)$: *存在*正常数c和n_0，使对所有的$n \geq n_0$，有 $0 \leq f(n) \leq c.g(n) \}$

即，对于任意一个函数f(n)，若存在正常数c，使*当n充分大*时，f(n)的值总在c.g(n)之下，则f(n)属于集合O (g(n))。

为了表示一个函数f(n)是集合O(g(n))的一个元素，记： f(n)=O(g(n))

直观图如右：



3.1 渐近符号

注意：（1）根据集合知识，有$\Theta(g(n))\subseteq O(g(n))$

（2）$an+b=O(n^2)$ 是上界，但不是最小上界。

（3）有些文献用O来非正式地描述渐近确界。现在文献中都将渐近确界和渐近上界区分开来。

（4）不要产生错误界限。

例如：$n^2+100n+6$，当$n<3$时，$n^2+100n+6<106n$，由此就认为 $n^2+100n+6=O(n)$. 事实上，对任何正的常数c，只要$n>c-100$就有$n^2+100n+6>c\times n$。

（5）用来作比较的函数g(n)应该尽量接近所考虑的函数f(n).

例如：$3n+2=O(n^2)$ 是松散的界限；$3n+2=O(n)$ 是较好的界限。同理，$3n^2+4\times 2^n=0(n^2)$是错误的界限。

（6）$f(n)=O(g(n))$不能写成$g(n)=O(f(n))$，因为两者并不等价。

前面提到，这里的等号并不是通常相等的含义。按照定义，用集合符号$\in$更准确些。

3.1 渐近符号

大O比率定理：对于函数f(n)和g(n)，如果极限 $\lim_{n\to\infty} f(n)/g(n)$ 存在，则f(n)=O(g(n))当且仅当存在正的常数c，使得 $\lim_{n\to\infty} f(n)/g(n) \le c$.

证明：略！

例如：因为 $\lim_{n\to\infty}\frac{3n+2}{n}=3$ ， 所以 $3n+2=O(n)$ ；

因为 $\lim_{n\to\infty}\frac{10n^2+4n+2}{n^2}=10$ ，所以 $10n^2+4n+2=O(n^2)$ ；

因为 $\lim_{n\to\infty}\frac{6*2^n+n^2}{2^n}=6$ ，所以 $6*2^n+n^2=O(2^n)$ ；

因为 $\lim_{n\to\infty}\frac{n^{16}+3*n^2}{2^n}=0$ ，所以 $n^{16}+3*n^2=O(2^n)$ ；

但是这不是一个好的上界估计，问题出在极限值不是正的常数。

注意： O记号用来表示上界，它表示对任意的输入，算法的最坏情况情况运行时间的上界。即最坏时间复杂性。

3.1 渐近符号

定理： 如果$f(n)=a_m n^m+ \ldots\ldots + a_1 n + a_0$ 且$a_m>0$,则$f(n)=O(n^m)$　证明：略！

下述不等式对于复杂性阶的评估非常有帮助：

定理： 对于任意一个正实数$x>0$和任一个实数$\varepsilon>0$,下面的结论都正确。

(1) 存在某个n_0使得对于任何$n\geq n_0$，有 $(\log n)^x < (\log n)^{x+\varepsilon}$ 。

例如 根据上面定理，我们很容易得出：

$n^3 + n^2 \log n = \Theta(n^3)$　　$\because$ $(\log n)^x<n$

$n^4 + n^{2.5} \log^{20} n = \Theta(n^4)$　　$\because$ $(\log n)^x<n$

$2^n n^4 \log^3 n + 2^n n^5 / \log^3 n = O(2^n n^5)$　$\because$ $(\log n)^x<n$

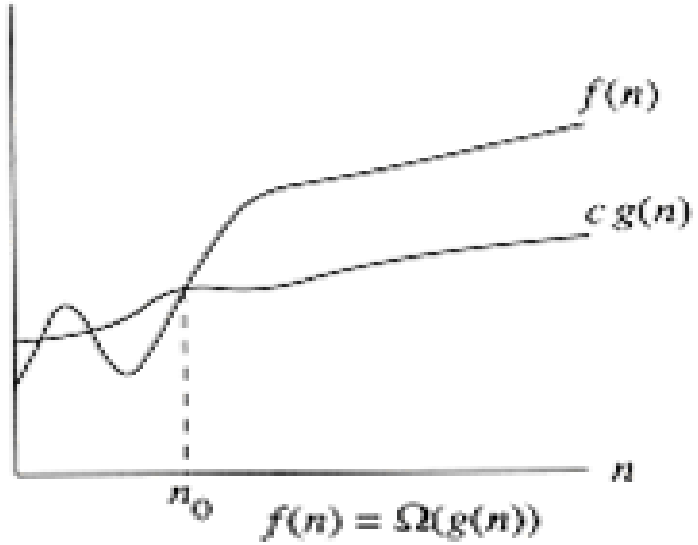
$n^x(\log n)^y<n^{x+\varepsilon}$

3.1 渐近符号

3. Ω记号—（渐近下界：lower bound)

Ω记号定义增长率的下界——一般是指最大下界

Ω(g(n)) = { f(n): 存在正常数c和n_0，使对所有的$n \geq n_0$，有 $0 \leq c.g(n) \leq f(n)$ }即，对于任意一个函数f(n)，若存在正常数c，使当n充分大时，f(n)的值等于或大于c.g(n)，则f(n)属于集合Ω(g(n))。直观表示如右图：



(c)

大Ω比率定理：对于函数f(n)和g(n)，如果极限 $\lim_{n\to\infty} g(n)/f(n)$存在，则f(n)= Ω(g(n))当且仅当存在正的常数c，使得 $\lim_{n\to\infty} g(n)/f(n) \leq c$

证明：略！

3.1 渐近符号

例如：$\lim_{n\to\infty} n/(3n+2)=1/3$ 所以 $3n+2=\Omega(n)$

$\lim_{n\to\infty} n^2/(10n^2+4n+2)=0.1$ 所以 $10n^2+4n+2=\Omega(n^2)$

$\lim_{n\to\infty} 2^n/(6*2^n+n^2)=1/6$ 所以 $6*2^n+n^2=\Omega(2^n)$

$\lim_{n\to\infty} n/(6n^2+2)=0$ 所以 $6*n^2+2=\Omega(n)$

$\lim_{n\to\infty} n^3/(3n^2+5)=\infty$ 所以 $3n^2+5\neq\Omega(n)$

定理： 如果$\mathbf{f(n)=a_m n^m+ \ldots\ldots + a_1 n + a_0}$ 且$\mathbf{a_m}>0$,则$f(n)=\mathbf{\Omega(n^m)}$

证明：略！

Ω记号用来表示渐近下界，它表示对任意的输入，算法的最好情况情况运行时间的下界。即最好时间复杂性。

3.1 渐近符号

4. o记号

O记号所提供的渐近上界可能是也可能不是渐近紧确的。例如，$2n^2=O(n^2)$是渐进紧确的，$2n=O(n^2)$就不是渐近紧确的。使用o符号来表示非渐近紧确的上界。即：

$f(n)= o(g(n))=$｛对任意正常数c，存在常数$n_0>0$，使得所有的$n\geq n_0$，$0\leq f(n)\leq c.g(n)$｝

O和o定义类似。主要区别是：

$f(n)=O(g(n))$，界$0\leq f(n)\leq c.g(n)$ 对*某个*常数$c>0$成立；

$f(n)= o(g(n))$，界$0\leq f(n)\leq c.g(n)$ 对*所有*常数$c>0$成立；

也就是说：在o表示中，当$n\to\infty$时，函数f(n)相对于g(n)来说就不重要了。即

$$\lim_{n\to\infty}\frac{f(n)}{g(n)}=0$$

3.1 渐近符号

5. ω记号

ω记号与Ω记号的关系就象o记号和O记号的关系一样。

$f(n)=\omega(g(n))=\{f(n)$:对任意正常数$c>0$，存在常数$n_0>0$，使得所有的$n\geq n_0$，有$0\leq c.g(n)<f(n)\}$

Ω和ω定义类似。主要区别是：

$f(n)=\Omega(g(n))$，界$0\leq c.g(n)<f(n)$对***某个***常数$c>0$成立；

$f(n)=\omega(g(n))$，界$0\leq c.g(n)<f(n)$对***所有***常数$c>0$成立；

也就是说：在ω表示中，当$n\to\infty$时，函数$g(n)$相对于$f(n)$来说就不重要了。即

$$\lim_{n\to\infty}\frac{f(n)}{g(n)}=\infty$$

3.1 渐近符号

实数的许多关系属性也一样适合渐近比较：

（1）传递性：

$f(n)=\Theta(g(n))$和$g(n)=\Theta(h(n))$ 则$f(n)=\Theta(h(n))$
$f(n)=O(g(n))$和$g(n)=O(h(n))$ 则$f(n)=O(h(n))$
$f(n)=\Omega(g(n))$和$g(n)=\Omega(h(n))$ 则$f(n)=\Omega(h(n))$
$f(n)=o(g(n))$和$g(n)=o(h(n))$ 则$f(n)=o(h(n))$
$f(n)=\omega(g(n))$和$g(n)=\omega(h(n))$ 则$f(n)=\omega(h(n))$

（2）自反性：

$f(n)=\Theta(f(n))$
$f(n)=O(f(n))$
$f(n)=\Omega(f(n))$

（3）对称性：

$f(n)=\Theta(g(n))$ 当且仅当 $g(n)=\Theta(f(n))$

（4）转置对称性：

$f(n)=O(g(n))$ 当且仅当 $g(n)=\Omega(f(n))$

$f(n)=o(f(n))$ 当且仅当 $g(n)=\omega(f(n))$

3.1 渐近符号

为了好理解，可以将两个函数f和g的渐近比较与两个实数的比较类比来看：

$f(n)= O(g(n))$ 类似于 $a \leq b$
$f(n)= \Omega(g(n))$ 类似于 $a \geq b$
$f(n)= \Theta(g(n))$ 类似于 $a = b$
$f(n)= o(g(n))$ 类似于 $a < b$
$f(n)= \omega(g(n))$ 类似于 $a > b$

注意：实数集上的“三分性”不适合渐近记号

对于两个实数a和b，下列三种情况恰有一种成立：a<b, a=b, a>b (这个性质称为“三分性”)但是，并不是所有函数都是渐近可比较的。即，对两个函数f(n)和g(n)，可能$f(n)= O(g(n))$和$f(n)= \Omega(g(n))$都不成立，当然$f(n)= \Theta(g(n))$也不成立。

例如：$f(n)=n$, $g(n)=n^{1+\sin n}$

3.1 渐近符号

多项式时间：**Polynomial-Time**

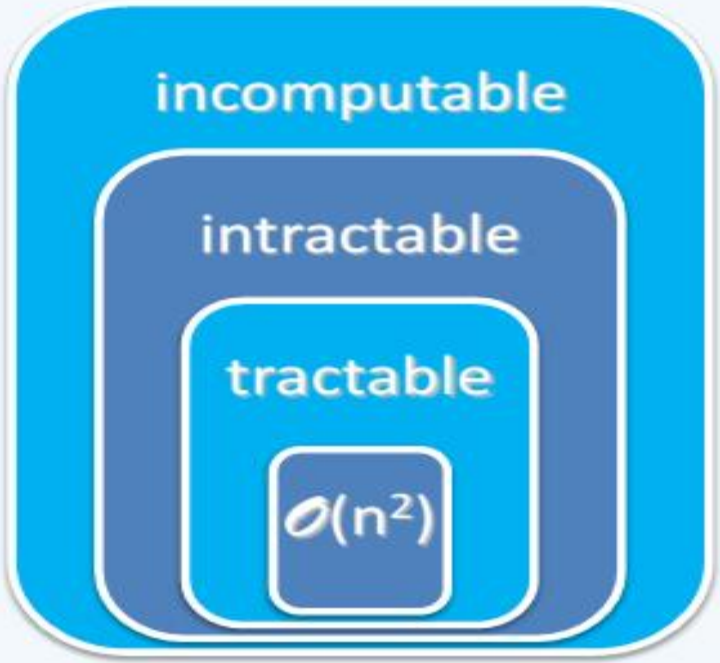
$T(n)=O(f(n))$， $f(n)$是多项式；

易解的（tractable），即能行可计算；

指数时间：**Exponential-Time**

$T(n)=O(a^n)$， $a>1$；

难解的（intractable）；



注意：在问题规模不大时，指数复杂度可能在较长的区间内好于多项式复杂度，但超越是必然的，而且随着n的进一步增大，二者的差距无法保持在多项式倍的范围！

3.2 算法分析的方法步骤

算法分析的一般步骤：

（1）确定问题规模参数；

（2） 确定关键（基本）操作、存储；

（3）建立各个量之间的“和”关系模型；

（4）利用数学方法求出累计和函数T(n),S(n)；

（5）给出渐近表示。

复杂度分析的主要方法

迭代：级数求和

递归：递归跟踪 + 递推方程

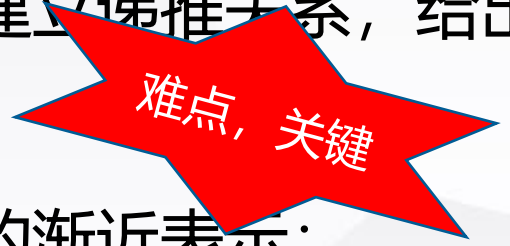
猜测 + 验证

3.2 算法分析的方法步骤

3.2.2 递归算法的时间复杂性分析

递归算法是比较特殊的一类算法，其分析过程一般有下面几步：

1. 根据递归算法 确定参数；
2. 分析递归算法，建立递推关系，给出边界条件，写出递归方程
3. 求解递归方程；
4. 给出递归方程解的渐近表示；



例1：HANOI塔求解问题

算法？

算法分析？

例2：二分查找

算法？

算法分析？

3.2 算法分析的方法步骤

3.2.2 递归算法的时间复杂性分析

递归方程：是一组等式或不等式，它所描述的函数是用在更小的输入下该函数的值来定义的。

在实际应用中，表达和解递归方程时常常略去一些技术性细节：

例如，数据结构中介绍的“归并”分类算法的最坏运行时间可以用下面的递归式来描述：

$$T(n)=\begin{cases}\Theta(1) & n=1\\ 2T(n/2)+\Theta(n) & n>1\end{cases}$$

其解为$T(n)=\Theta(n\log n)$

在这里，递归式实际应该是：

$$T(n)=\begin{cases}\Theta(1) & n=1\\ T(\lceil n/2\rceil)+T(\lfloor n/2\rfloor)+\Theta(n) & n>1\end{cases}$$

另外，边界条件n=1的情况可以省略！

3.2 算法分析的方法步骤

3.2.2 递归算法的时间复杂性分析

递归算法在*最坏情况下*的时间复杂性渐近阶的分析，都转化为数学中求相应的一个递归方程的解的渐近阶问题。因此，求递归方程的解的渐近阶是对递归算法进行分析的关键步骤。

递归方程的形式多种多样，求其解的渐近阶的方法也多种多样。常见的比较实用的方法有5种：

替换法（Substitution Method）

递归树（迭代法） (Iteration Method)

套用公式法 (Master Method)

差分方程法 (Difference Formula Method)

母函数法 (Cteneral Functions Method)

3.2 算法分析的方法步骤

3.2.3 递归方程求解之：替换法

这是一种使用*数学归纳法*推导证明的方法，其步骤为先假设一个解，然后带入到递归方程中，利用数学归纳法推导，以验证假设的解是否合理。

它可用来确定一个递归式的上界或下界。这种方法很有效，但是只能**用于解的形式很容易猜的情形**。

1. 替换法解递归式的步骤：

- 猜测一个答案；
- 使用数学归纳法对答案进行验证（根据猜测的答案形成归纳假设，然后对归纳假设进行验证）；
- 解决表达式中的常数；

3.2 算法分析的方法步骤

3.2.3 递归方程求解之：替换法

例如1，有下面的递归式： T(n)=2T(⌊n/2⌋)+n，我们可以用代换法来确定它的一个上界。其中是⌊ ⌋表示不大于n的最大整数。

我们推测T(n)=O(n*logn*)，即推测存在正的常数c和自然数n_0，使得当$n \geq n_0$时有：

$T(n) \leq cn\log n$

递归基础：事实上，取$n_0=2^2=4$，并取 $C = \max_{n_0<n<2n_0} (T(n)/(n\log n))+1$

归纳假设：

今归纳假设当$2^{k-1}n_0 \leq n \leq 2^k n_0$，$k \geq 1$时，$T(n) \leq cn\log n$成立。

归纳证明：当$2^k n_0 \leq n \leq 2^{k+1} n_0$时，$T(n) \leq cn\log n$是否成立 。

3.2 算法分析的方法步骤

因为：

$$
\begin{aligned}
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2C\lfloor n/2 \rfloor \cdot \log(\lfloor n/2 \rfloor) + n \\
&< 2C \cdot \frac{n}{2} \cdot \log \frac{n}{2} + n \\
&= Cn\log n - Cn + n \\
&= Cn\log n - (c-1)n \\
&\le Cn\log n
\end{aligned}
$$

即仍然成立 。于是对所有$n \ge n_0$，$T(n) \le cn\log n$ 成立。可见我们的推测是正确的。因而得出结论：递归方程的解的渐近阶为$O(n\log n)$。

3.2 算法分析的方法步骤

这个方法的局限性在于它只适合容易推测出答案的递归方程或善于进行推测的高手。推测递归方程的正确解，没有一般的方法，得靠经验的积累和洞察力。在这里给出三点有益处的建议：

(1) 如果一个递归方程类似于你从前见过的已知其解的方程，那么推测它有类似的解是合理的。

例如，考虑递归方程： $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$

右边项的变元中加了一个数17，使得方程看起来难于推测。但是它在形式上与前面的 例1 的递归方程很类似。实际上，当n充分大时 $T(\lfloor n/2 \rfloor + 17) + n$ 和 $T(\lfloor n/2 \rfloor)$ 相差无几。因此可以推测与前面递归方程有类似的上界$T(n)=O(n\log n)$。进一步，数学归纳将证明此推测是正确的。

3.2 算法分析的方法步骤

3.2.3 递归方程求解之：替换法

(2) 从较宽松的界开始推测，逐步逼近精确界。

比如对于例1中的递归方程，要估计其解的渐近下界。由于明显地有$T(n)\geq n$，我们可以从推测$T(n)=\Omega(n)$开始，发现太松后，把推测的阶往上提，就可以得到$T(n)=\Omega(n\log n)$的精确估计。

(3) 对变元作替换，使一个未知其解的递归方程变成类似于曾见过的已知其解的方程，得到变换后方程的解后，再作逆变换，便可得到所需要的解。

例如，考虑递归方程：

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log n$$

看起来很复杂，因为右端变元中带根号。但是，如果作变元替换$m=\log n$，即令$n=2^m$，将其代入，则变成：

3.2 算法分析的方法步骤

3.2.3 递归方程求解之：替换法

$$T(2^m) = 2T(\lfloor 2^{m/2} \rfloor) + m \qquad (6.5)$$

把m限制在正偶数集上，则 又可改写为：　　$T(2^m)=2T(2^{m/2})+m$

若令$S(m)=T(2^m)$，则$S(m)$满足的递归方程：　　$S(m)=2S(m/2)+m$

与例1类似，因而有：

$$S(m)=O(m\log m)$$

进而得到$T(n)=T(2^m)=S(m)=O(m\log m)=\boldsymbol{O(\log n\log\log n)}$

尽管 上面证明的 只是：当(充分大的)n是2的正偶次幂或换句话说是4的正整数次幂时才成立。但可以推出对所有充分大的正整数n也都成立，从而，递归方程解的渐近阶得到估计。

3.2 算法分析的方法步骤

3.2.4 递归方程求解之：迭代法（递归树）

从上面例子可见迭代法导致繁杂的代数运算。但是，迭代使递归表达式的抽象变的具体了，我们更喜欢具体的东西，因为抽象有时让我们看不清楚里面隐含的具体执行和规律，例如：递归成本。

如果通过图形的方式把抽象表达式展开，则抽象变具体，就容易理解多了。虽然不是所有的抽象都可以用具体的形式来描述，但是抽象的递归式却可以被具体化。

使用递归树，可以使迭代法的步骤直观简明、图表化，递归树给出了一个算法递归执行的代价模型。

3.2 算法分析的方法步骤

3.2.4 递归方程求解之：迭代法（递归树）

递归树的构造方法是：

该树的树根结点是顶层递归的代价，根的子树是根据递归式推出的更小的递归式。即首先T(n)扩展为一棵等价的树，然后再依次下去，继续扩展递归式的各个结点，即将其分解成由递归式所决定的各个组成部分，直到问题规模降到了最小。

在递归树中，每个结点都代表递归函数调用集合中一个子问题的代价。将树中每一层的代价相加得到一个每层代价的集合，再将每层的代价相加得到递归式所有层次的总代价。

例1，递归方程： $T(n)=2T(n/2)+n^2$ (1)

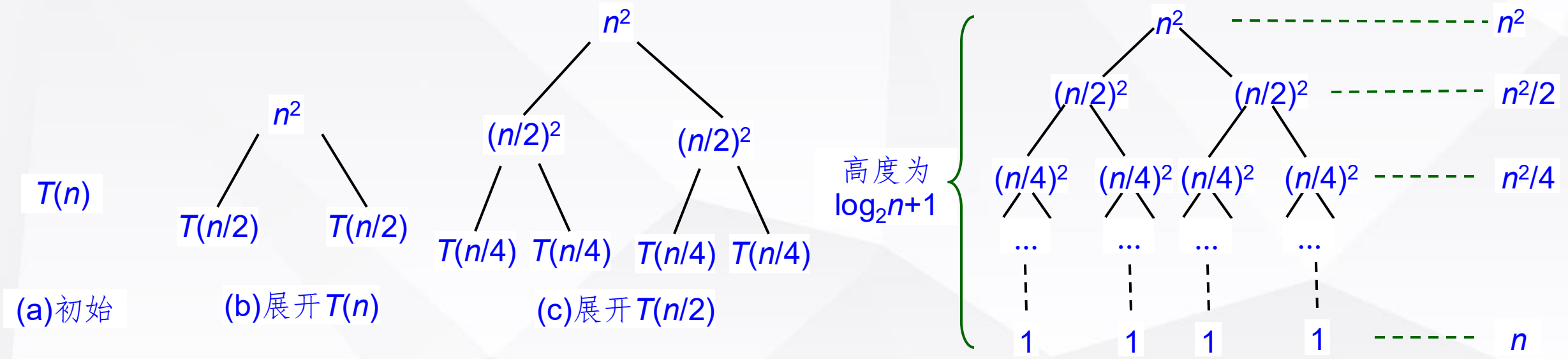
递归树构造如下：

3.2 算法分析的方法步骤

【例】分析以下递推式的时间复杂度。

$$T(n)=1 \quad \text{当} n=1$$
$$T(n)=2T(n/2)+n^2 \quad \text{当} n>1$$

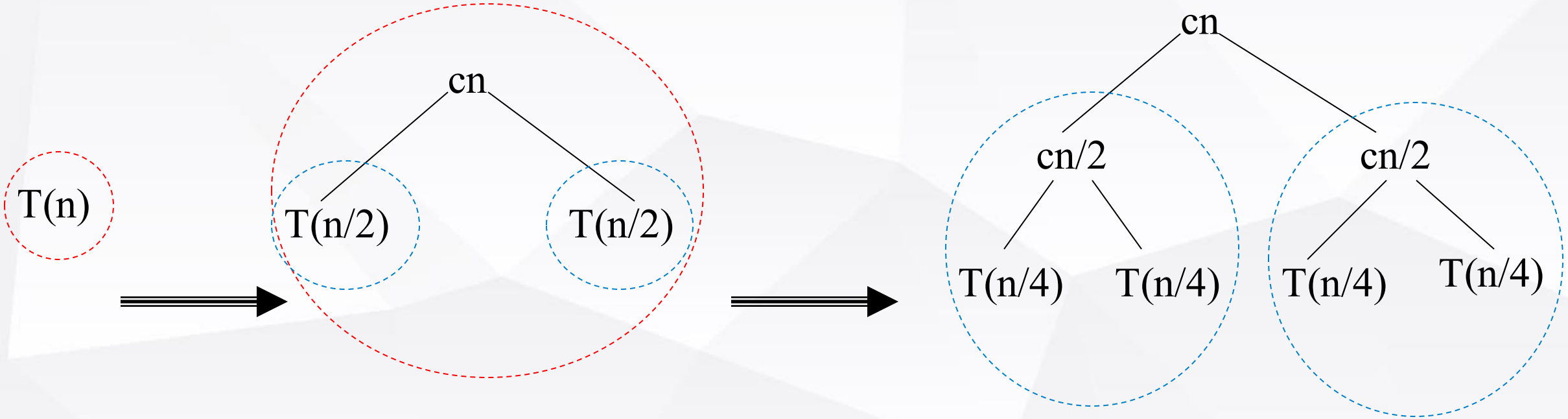
解



- 第1层的问题规模为n，第2的层的问题规模为$n/2$，依此类推，当展开到第$k+1$层，其规模为$n/2^k=1$，所以递归树的高度为$\log_2 n+1$。
- 所有层的结点个数$T(n)=n^2+n^2/2+\ldots+n^2/2^{k-1}+\ldots+n=O(n^2)$。

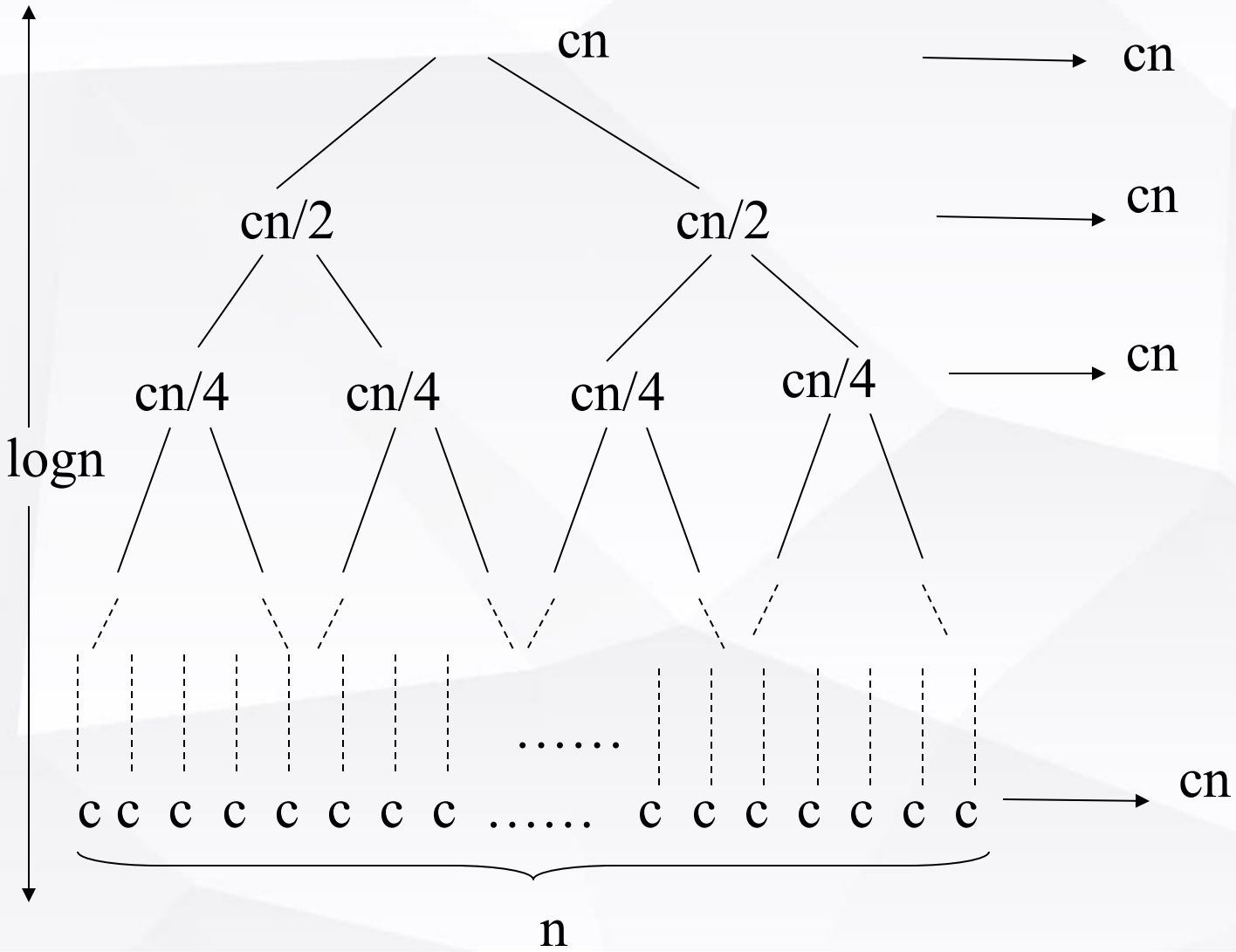
3.2 算法分析的方法步骤

例2，对于递归式 T(n)＝2T(n/2)+cn，递归树构造为：



3.2 算法分析的方法步骤

3.2.4 递归方程求解之：迭代法（递归树）



当我们累计递归树各层的值时，得到每一层的和都等于n，从根到叶的最长路径是：

$$n \to \frac{1}{2}n \to \frac{1}{2^2}n \to \ldots \to 1$$

设最长路径的长度为k，则应该有：$\left(\frac{1}{2}\right)^k = 1$，得：$k=\log_2 n$

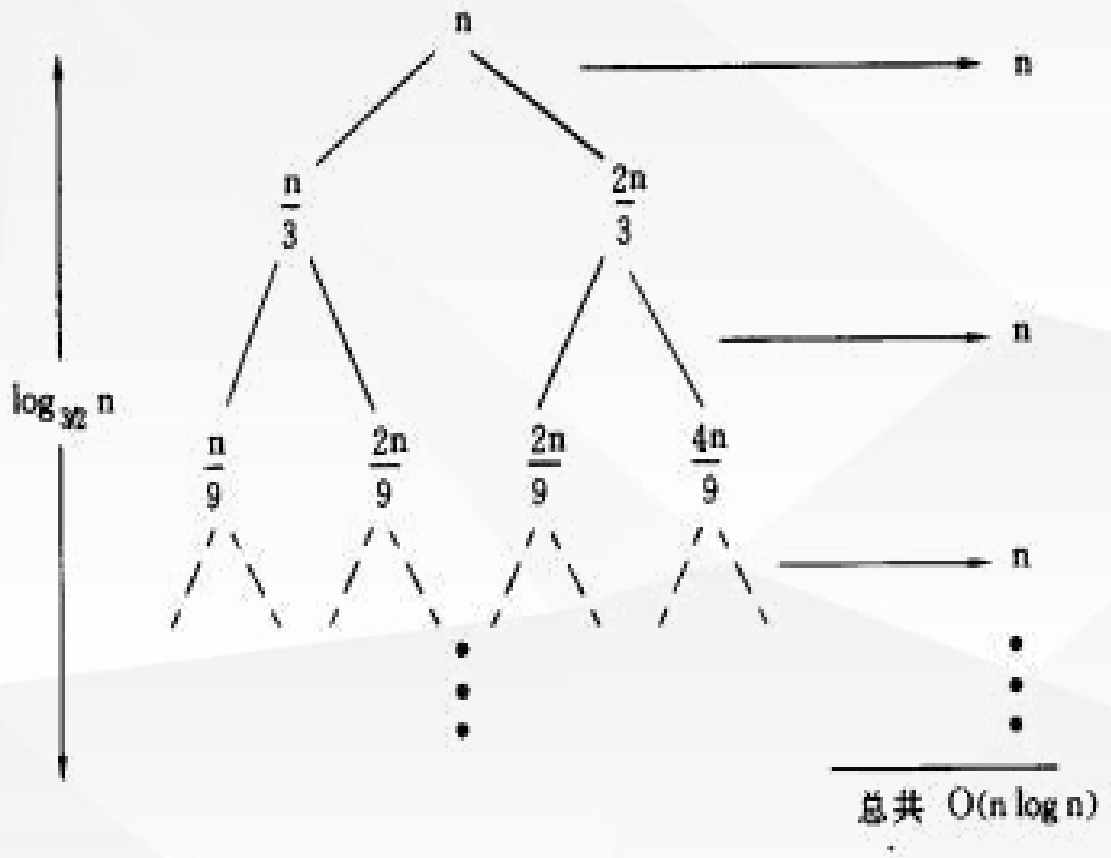
$$T(n) \le \sum_{i=0}^{k} n = (k+1)n = n(\log_2 n + 1)$$

即$T(n)=O(n\log n)$。

3.2 算法分析的方法步骤

3.2.4 递归方程求解之：迭代法（递归树）

例3，递归方程：$T(n)= T(n/3)+ T(2n/3)+n$其相应的递归树。 **（自己独立完成）**



当我们累计递归树各层的值时，得到每一层的和都等于n，从根到叶的最长路径是：

$$n \to \frac{2}{3}n \to (\frac{2}{3})^2 n \to \cdots \to 1$$

设最长路径的长度为k，则应该有：$(\frac{2}{3})^k n = 1$ ，得：

$$k = \log_{3/2} n$$

于是

$$T(n) \le \sum_{i=0}^{k} n = (k+1)n = n(\log_{3/2} n + 1)$$

即$T(n)=O(n\log n)$。

3.2 算法分析的方法步骤

3.2.4 递归方程求解之：迭代法（递归树）

递归树最适合来产生好的猜测，然后用代换法加以验证。但是，在用递归树产生好的猜测时，由于后面还要对猜测进行归纳证明，所以产生递归树时可以是一个大致的粗略的，即不一定是非常严格的全部生成出来。

如果生成递归树时非常仔细完整，并且将代价都加了起来，那么就可以直接用递归树来作为递归式解的证明，这也就是我们经常用的展开法求解递归式的解。

练习：

请画出递归式： $T(n)=T(n/4)+T(n/2)+n^2$的递归树，并给出解的O表示。

3.2 算法分析的方法步骤

3.2.5递归方程求解之：套用公式法

Master method (套用公式法，大师解法)给出求解如下形式的递归式的“食谱”方法。

$$T(n)=aT(n/b)+f(n)$$ 其中 a≥1和b>1是常数，f(n)是一个渐近正的函数。

上面的递归式描述了将规模为n的问题划分为a个子问题，每个子问题的规模为n/b，a和b是正常数，a个子问题被递归地解决，其花时间为T(n/b)。划分原问题与合并答案的代价用函数f(n)来描述。

大师解法要求只要记忆三种情况，就可以很容易地确定一些递归式的解，而且不用纸和笔！

3.2 算法分析的方法步骤

3.2.5递归方程求解之：套用公式法

Master定理：

设$a \geq 1, b > 1$为常数，$f(n)$为函数

$$T(n) = aT(n/b) + f(n),$$

$T(n)$为非负整数

1. $f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0,$

   那么$T(n) = \Theta(n^{\log_b a})$

2. $f(n) = \Theta(n^{\log_b a}),$

   那么$T(n) = \Theta(n^{\log_b a} \log n)$

3. $f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0,$

   且对于某个常数$c < 1$和所有的充分大的n有

   $$af(n/b) \leq cf(n),$$

   那么$T(n) = \Theta(f(n))$

3.2 算法分析的方法步骤

3.2.5递归方程求解之：套用公式法

这里涉及的三类情况，都是拿$f(n)$与$n^{\log_a b}$作比较。定理直观地告诉我们，递归方程解的渐近阶由这两个函数中的较大者决定。

在第一类情况下，函数$n^{\log_a b}$较大，则$T(n)=\Theta(n^{\log_a b})$；

在第三类情况下，函数$f(n)$较大，则$T(n)=\Theta(f(n))$；

在第二类情况下，两个函数一样大，则$T(n)=\Theta(n^{\log_a b}\cdot\log n)$，

即以n的对数作为因子乘上$f(n)$与$T(n)$的同阶。

3.2 算法分析的方法步骤

3.2.5递归方程求解之：套用公式法

注意：

在第一类情况下$f(n)$不仅必须比 $n^{\log_a b}$ 小，而且必须是多项式地比 $n^{\log_a b}$ 小，即$f(n)$必须渐近地小于$n^{\log_a b}$与 $n^{-\varepsilon}$ 的积，ε是一个正的常数；

在第三类情况下$f(n)$不仅必须比 $n^{\log_a b}$ 大，而且必须是多项式地比 $n^{\log_a b}$ 大，还要满足附加的“正规性”条件：$af(n/b)\leq cf(n)$。

这个附加的“正规性”条件的直观含义是：a个子问题的再分解和再综合所需要的时间最多与原问题的分解和综合所需要的时间同阶。

还有一点很重要，即要认识到上述三类情况并没有覆盖所有可能的$f(n)$。

在第一类情况和第二类情况之间有一个间隙：$f(n)$小于但不是多项式地小于$n^{\log_a b}$；类似地，在第二类情况和第三类情况之间也有一个间隙：$f(n)$大于但不是多项式地大于$n^{\log_a b}$。如果函数$f(n)$落在这两个间隙之一中，或者虽有$f(n)=\Omega(n^{\log_b a+\varepsilon})$，但正规性条件不满足，*那么，本定理无能为力。*

3.2 算法分析的方法步骤

3.5.5递归方程求解之：套用公式法

主定理的应用：

例1： $T(n)=9T(n/3)+n$ ，这里，$a=9, b=3$ ， $f(n)=n$。

由于 $n=O(n^{\log_3 9-1})$ ，$\varepsilon=1$ 所以由公式(I)知： $T(n)=\Theta(n^{\log_3 9})=\Theta(n^2)$

例2： $T(n)=T(2n/3)+1$

这里 $a=1$， $b=3/2$， $f(n)=1$

由于 $1=O(n^0)=O(n^{\log_{3/2} 1})$ ，$f(n)=\Theta(n^{\log_b a})=\Theta(1)$

所以由公式(II)有 $T(n)=\Theta(\lg n)$ 。

3.2 算法分析的方法步骤

3.2.5递归方程求解之：套用公式法

例3： T(n)=3T(n/4)+nlogn

请同学们自己完成！

对照定理，我们有a=3，b=4， f(n)=nlog n，$n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$

只要取 $\varepsilon \approx 0.2$ ，便有 $f(n) = \Omega(n^{\log_b a + \varepsilon})$

进一步，检查正规性条件：

$$af(n/b) = 3f(n/4) = 3(n/4)\log(n/4) = \frac{3}{4}n(\log n - \log 4) \le \frac{3}{4}n\log n$$

只要取c=3/4，便有af(n/b)≤cf(n)，即正规性条件也满足。可套用第三类情况的公式，得T(n)=θ(f(n))=θ(nlogn)。

例： $T(n)=2T(n/2)+n\log n$

这是一个本方法对之无能为力的例子

考虑递归方程： $T(n)=2T(n/2)+n\log n$

对照定理式，我们有$a=2$，$b=2$， $f(n)=n\log n$，$n^{\log_b a}=n$

虽然$f(n)$渐近地大于 $n^{\log_a b}$

但$f(n)$并不是多项式地大于 $n^{\log_a b}$

因为对于任意的正常数ε，

$$f(n)/n^{\log_b a+\varepsilon}=n\log n/n^{1+\varepsilon}=n^{-\varepsilon}\log n\to 0$$

即$f(n)$在第二类情况与第三类情况的间隙里，本方法对它无能为力。

3.2 算法分析的方法步骤

3.2.6 递归方程求解之：差分方程法

有些递归方程可以看成一个差分方程，因而可以用解差分方程(初值问题)的方法来解递归方程。然后对得到的解作渐近阶的估计。

只考虑形如：

$T(n)=c1T(n-1)+c2T(n-2)+\ldots+ ckT(n-k)+f(n)$，$n\geq k$的递归方程。其中ci (i=1，2，…，$k$)为实常数，且$ck\neq 0$。

它可改写为一个线性常系数k阶非齐次的差分方程：

$T(n)-c1T(n-1)- c2T(n-2)-\ldots-ckT(n-k)=f(n)$，$n\geq k$

它与线性常系数k阶非齐次常微分方程的结构十分相似，因而解法类同。

——有兴趣的自学

3.2 算法分析的方法步骤

3.2.7 递归方程求解之：母函数法

这是一个有广泛适用性的方法。它不仅可以用来求解线性常系数高阶齐次和非齐次的递归方程，而且可以用来求解线性变系数高阶齐次和非齐次的递归方程，甚至可以用来求解非线性递归方程。

方法的基本思想是设定递归方程解的母函数，努力建立一个关于母函数的可解方程，将其解出，然后返回递归方程的解。

——有兴趣的自学

3.6 算法分析举例

例3：

```
算法 BS(A,i,j,fmax,fmin)
//？？
BS1 [递归出口]
    if(i==j) { fmax ← fmin←a[i];return }
    if(i==j-1)
      { if(a[i]<a[j]) { fmax ←a[j]; fmin ←a[i] }
          else { fmax ←a[i]; fmin ←a[j] }
        return }
BS2 [取中值]
    mid ←[(i+j)/2]
BS3 [递归调用]
    BS(A,i,mid,gmax,gmin);
    BS(A,mid+1,j,hmax,hmin)
BS4 [合并]
    fmax ←max{gmax,hmax}
    fmin ←min{gmin,hmin}
END
```

$$T(n)=\begin{cases}0 & n=1\\ 1 & n=2\\ 2T(n/2)+2 & n>2\end{cases}$$

设$n=2^k$

则：$T(n)=2T(n/2)+2$

$=2*[2T(n/4)+2]+2$

$=2*\{2*[2T(n/8)+2]+2\}+2$

$=\cdots\cdots\cdots$

$=2^{k-1}*T(2)+\sum 2^i$

$=2^{k-1}+2^k-2$

$=3/2n-2$

3.6 算法分析举例

倍率实验— 简单有效预测任意程序性能
判断所用算法大致增长数量级（阶）

1. 倍率定理

如果 $T(n) \sim an^b\log n$

则，$T(2n)/T(n) \sim 2^b$ （指数级别除外）

2. 方法

（1）设计一个输入生成器，能产生实际情况的各种可能输入
最常用的是使用随机函数；
（2）让输入规模不断加倍，并计算本次运行时间和上次运行时间的比值；
（3）反复运行，直到该比值趋近于极限2^b；（如果没有极限，则该方法无效）
（4）**粗略得到算法的增长率为N^b；**

3.6 算法分析举例

倍率实验— 简单有效预测任意程序性能
判断所用算法大致增长数量级（阶）

有什么用？

在有性能压力的情况下，应考虑对编写的算法进行倍率实验，或许能发现性能上的问题，例如，你的程序可能并没有你想象的那么高效！

- **评估它解决大型问题的可行性**

“该程序能在可接受的时间内处理这些数据吗？”

——大数据处理中常见的困惑

- **评估使用更快的计算机所产生的价值**

“如果我能够得到一台更快的计算机，解决问题的速度能加快多少？”

——硬件更新中常关心的问题

谢谢大家